SETUP GUIDE

CRM Sync — PWA & Native App Commerce Setup Guide

Ship your CRM as an installable **PWA** and **native app** (iOS · Android · desktop) on Shopify + Webflow + Xano — auth, real-time consent, GA4, and agent permissions. Six services — Worker, Xano, Shopify, GA4, Resend, Webflow — configured in order; each section builds on the last.

Ship your CRM as an installable **PWA** and **native app** (iOS · Android · desktop) on Shopify + Webflow + Xano — with auth, real-time consent, GA4, and agent permissions. Everything you need to get it running; complete each section in order — each one builds on the last.

> **What is CRM Sync?** It connects six services together: a **server** (Cloudflare Worker) that runs your auth and sync logic, a **database** (Xano) that stores users and tag relationships, a **store** (Shopify) for customer sync, **analytics** (Google GA4) for tracking consent and user segments, **email** (Resend) for transactional emails, and a **CMS** (Webflow) that displays user data and manages campaigns. You configure each one, then the Webflow App ties them together.

CRM Sync PIM Sync Shopify Checklist [Strategy] (https://miro.com/app/board/uXjVHDMJNpA=/?share_link_id=653901982235) Accessibility & Machine-Index Spec Dawn → Horizon + GraphQL/GA4 Brief

---

Tab 1: Cloudflare Worker

What is this?

The Cloudflare Worker is your **backend server**. It handles:

- **User authentication** — signup, login, password reset, OAuth (Google + Shopify)
- **Consent management** — cookie banners, TOS acceptance, GDPR compliance
- **Customer sync** — bidirectional tag sync between Shopify, Xano, Webflow CMS, and GA4
- **Tag system** — structured CRM tags with categories (status, tier, segment, campaign, consent, marketing)
- **GA4 integration** — pushes user properties and events to Google Analytics via Measurement Protocol
- **Serving UI** — the login modal, consent banner, account page, and UCP dashboard are all served from here

Why Cloudflare? It's fast (runs at the edge, close to your users), has a generous free tier, and includes KV storage for caching config and session data.

▶ **Setup Steps**

Alternative: Skip the CLI

If you install the CRM Sync Webflow extension, you can set all credentials from the **Config** tab in the Webflow Designer panel. Values saved there override `wrangler.toml` — so after the initial deploy, you never need the command line again.

Tab 2: Xano (Database)

What is this?

Xano is your **database**. It stores all your user accounts, consent records, profile data, and the CRM tag system. The CRM Worker talks to Xano using the

Meta API.

What gets stored:

- **User accounts** — email, name, password hash, login provider, Shopify/Google IDs
- **Consent preferences** — which policies each user accepted, when, and how
- **Consent audit log** — a timestamped record of every consent change (required for GDPR)
- **CRM tags** — structured tags with name, slug, and category
- **User-tag assignments** — a join table linking users to tags with source attribution
- **User extras** — a flexible table for any additional data

▶ **Setup Steps**

Tab 3: Shopify (Store Integration)

What is this?

The Shopify integration does four things:

1. **Admin API** — Syncs customer data between your CRM and Shopify. Tags, metafields, consent status, and order data are synced in real-time via webhooks and every 15 minutes via cron. Also handles GDPR data requests.
2. **Real-Time Webhooks** — When a customer is created or updated in Shopify, webhooks immediately sync to Xano and Webflow CMS. New Shopify customers automatically receive a welcome email to set their website password.

3. **Customer Account OAuth** — Lets users sign in with their Shopify account using PKCE (no client secret needed).
4. **CRM Tag Sync** — Tags added from the UCP dashboard flow to Shopify as both customer tags (for Shopify Segments/Flows) and structured metafields (for custom reporting).

You need three things from Shopify: a Dev Dashboard app (for Admin API token + app secret), a Headless channel (for OAuth login), and GDPR webhook URLs.

▶ **Setup Steps: Dev Dashboard App (Admin API)**
▶ **Setup Steps: "Sign in with Shopify" (OAuth)**
▶ **Setup Steps: GDPR Compliance Webhooks**

Tab 4: Google (Analytics, OAuth & GA4 Segments)

What is this?

The Google integration has three parts:

1. **Google OAuth** — "Sign in with Google" button via OpenID Connect.
2. **GA4 Consent Mode** — Connects your CRM consent banner to Google Analytics. When a user accepts or rejects cookies, GA4 is updated to respect their choice.
3. **GA4 Measurement Protocol** — Server-side push of CRM user properties to GA4. Every tag change (from the UCP dashboard, Shopify sync, or admin API) pushes structured user properties to GA4, making them available for GA4 audiences, Google Ads audience sharing, and Looker Studio.

GA4 User Properties pushed by the worker:

PROPERTY	SOURCE	EXAMPLE VALUE
`crm_status`	Status category tags	`active`
`crm_tier`	Tier category tags	`vip`
`crm_segment`	Segment category tags	`high_value,returning`
`crm_campaign`	Campaign category tags	`new_campaign,summer_2026`
`crm_tags`	All tag slugs	`active,vip,new_campaign`
`consent_marketing`	Consent tags	`granted` or `denied`
`consent_tos`	Consent tags	`granted` or `denied`

Events sent:

- `crm_tags_updated` — fired when a user adds/removes tags from the dashboard (includes `tags_added`, `tags_removed`, `campaign_tags` params)
- `crm_sync` — fired during the 15-minute cron sync from Shopify (includes `source: shopify_cron`)

▶ **Setup Steps: Google OAuth ("Sign in with Google")**

▶ **Setup Steps: GA4 Measurement Protocol (Server-Side Segments)**

▶ **Setup Steps: GA4 Consent Mode (Client-Side)**

▶ **Google Tag Manager — not required**

▶ **CRM Form Bridge: E2E Event Tracking (Any Form Type)**

Tab 5: Webflow CMS (Customer Data & Tags)

What is this?

Webflow CMS stores a read-friendly copy of your customer data and CRM tags. This lets you build Webflow pages that display customer profiles, filter by tags, and create dynamic content based on CRM segments.

Two collections are used:

1. **Customers** — synced from Xano on every cron run (name, email, provider, consent status, order data, tags)
2. **CRM Tags** — the tag definitions with categories, referenced by the Customers collection via MultiReference

▶ **Setup Steps**

Campaign Tags from Dashboard

When a user adds a tag like "new campaign" from the UCP dashboard,
it immediately:

1. Creates the tag in Xano (`crm_tags` table, category: `campaign`)
2. Assigns it to the user in the join table (`user_tag_map`)
3. Pushes it to Shopify as a customer tag + metafield
4. Creates it in the Webflow CRM Tags collection
5. Pushes it to GA4 as a `crm_campaign` user property

No cron wait — the full channel flow happens in one request.

Tab 6: Email (Resend)

What is this?

Resend handles **transactional emails**:

1. **Password reset** — When a user clicks "Forgot password?", the worker sends a reset link (1-hour expiry).
2. **Welcome email** — When a customer is added in Shopify and synced to the CRM, they automatically receive a "Welcome — set up your password" email (24-hour expiry). This lets Shopify-origin users create a password to sign in on the website. The forgot-password flow also detects first-time users and sends the welcome variant instead of the reset variant.

▶ **Setup Steps**

Tab 7: Webflow Extension (Config UI)

What is this?

The CRM Sync Webflow extension adds a configuration panel to the Webflow Designer. It lets you manage all credentials, auth settings, consent toggles, and embed codes without touching the CLI.

Config tab — Worker URL, Shopify credentials, Google OAuth, Xano, Resend, Webflow CMS, GA4 **Auth tab** — Toggle auth methods (email/Google/Shopify), session settings, consent & privacy toggles **Embeds tab** — Copy-paste embed codes for footer loader, account page, dashboard, compliance page **Status tab** — Health check, endpoint testing, redirect URIs, privacy API tests, GDPR handler tests

Embed Codes

The extension generates four embed snippets:

EMBED	WHERE TO PASTE	WHAT IT RENDERS
CRM Footer Loader	Site Settings > Footer Code	Login modal, consent banner, session management
Account Page	Account page embed block	User profile view/edit
UCP Dashboard	Dashboard page embed block	Consent status, retarget channels, A/B segment, tags, translation, consent history
Compliance / Privacy	Privacy page embed block	Communication preferences, third-party disclosures, data rights

Tab 8: A2A — Agent-to-Agent Protocol

What is this?

A2A (Agent-to-Agent) is Google's open protocol for AI agents to discover and communicate with each other. CRM Sync implements A2A so that external AI agents — shopping assistants, support bots, recommendation engines — can programmatically discover your store's capabilities, read customer profiles, browse products, and check order history.

Two discovery endpoints:

1. **UCP Discovery Manifest** (`GET /.well-known/ucp`) — Declares all capabilities the worker supports (identity linking, consent management, product catalog, checkout, orders, analytics, etc.). Capabilities are generated dynamically based on which services are configured for the tenant.

2. **A2A Agent Card** (`GET /.well-known/agent-card.json`) — A standard agent card that AI platforms use to discover CRM Sync as a commerce service agent. Declares 5 skills: Identity Linking, Consent Management, Customer Tag Sync, Product Catalog, and Checkout Session.

Commerce API endpoints exposed via A2A:

ENDPOINT	METHOD	AUTH	WHAT IT DOES
`/commerce/identity`	GET	JWT	Unified customer profile with linked providers (email, Google, Shopify), consent status, tags, segment
`/commerce/products`	GET	Public	Search product catalog with pagination and collection filtering
`/commerce/orders`	GET	JWT	List customer's order history (cached 5 min)
`/commerce/orders/:id`	GET	JWT	Single order detail with line items and fulfillment status
`/ucp/consent-history`	GET	JWT	Paginated consent audit trail
`/ucp/tags`	POST	JWT	Add/remove customer tags (flows to Shopify, Webflow CMS, GA4)

Customer consent required: A2A access is gated behind the `consent_a2a` flag. Customers must explicitly enable "A2A Agent Access" in their account preferences before any AI agent can read their profile or commerce data. The consent toggle appears in the Account page and Dashboard embeds.

When A2A consent is enabled, the UCP manifest dynamically adds `a2a_agent_access` to its capabilities list.

▶ **Setup Steps**

Agent Card Schema

The Agent Card at `/.well-known/agent-card.json` follows the A2A spec:

```
{
  "name": "CRM Sync Commerce Agent",
  "description": "Multi-tenant customer identity, consent, and commerce orchestration across 7
  "version": "1.0",
  "protocol_version": "0.1",
  "capabilities": {
    "streaming": false,
    "pushNotifications": false,
    "stateTransitionHistory": true
  },
  "authentication": {
    "schemes": ["bearer"],
    "credentials": {
      "bearer": {
        "description": "JWT token from /auth/login, /auth/google/callback, or /auth/shopify/cal
      }
    }
  },
  "skills": [
    { "id": "identity-linking", "name": "Identity Linking", ... },
    { "id": "consent-management", "name": "Consent Management", ... },
    { "id": "tag-sync", "name": "Customer Tag Sync", ... },
    { "id": "product-catalog", "name": "Product Catalog", ... },
    { "id": "checkout-session", "name": "Checkout Session", ... }
  ]
}
```

Tab 9: AP2 — Agent Payments Protocol

What is this?

AP2 (Agent Payments Protocol) ensures that AI agents **cannot make purchases without explicit human authorization**. It extends the A2A protocol with cryptographically signed payment mandates — a customer must create a mandate (spending limit + scope + expiry) before any agent can initiate a checkout on their behalf.

How it works:

```
Customer enables AP2 consent in account preferences
  → Customer creates a mandate: "Up to $100 for any product, expires in 24h"
  → Worker generates HMAC-SHA256 signed mandate, stores in KV
  → AI agent calls POST /commerce/checkout with mandate_id
  → Worker validates: mandate is active, not expired, covers the purchase amount/scope
  → If valid: Shopify checkout created via Storefront Cart API
  → If invalid: 403 with reason (expired, exceeded, wrong scope)
  → Mandate usage logged to consent audit trail + GA4
```

AP2 consent gate: If `consent_ap2` is not granted, `POST /commerce/mandates` returns `403 "AP2 agent payments consent required"`. The customer must enable it first.

Endpoints:

ENDPOINT	METHOD	AUTH	WHAT IT DOES
`/commerce/mandates`	POST	JWT	Create a new payment mandate
`/commerce/mandates/:id`	GET	JWT or Admin	Retrieve mandate status and details
`/commerce/checkout`	POST	JWT	Create checkout session (optionally with `mandate_id`)
`/commerce/checkout/:id`	GET	JWT or Admin	Get checkout session status

Mandate schema:

```
{
  "mandate_id": "uuid",
  "customer_id": 123,
  "type": "purchase",
  "max_amount": "100.00",
  "currency": "USD",
  "scope": "all | collection:summer-2026 | product:gid://shopify/Product/123",
  "status": "active | used | expired | revoked",
  "created_at": "2026-05-24T10:00:00Z",
  "expires_at": "2026-05-25T10:00:00Z",
  "signature": "base64-hmac-sha256"
}
```

GA4 events fired:

- `ucp_mandate_created` — when a mandate is created (includes `mandate_id`, `max_amount`, `currency`, `scope`)
- `ucp_mandate_used` — when a mandate is used for checkout
- `ucp_checkout_created` — when a checkout session is created

- `ucp_checkout_completed` — when a checkout session completes

▶ Setup Steps

Security Model

CONTROL	IMPLEMENTATION
Consent gating	`consent_ap2` must be `true` before any mandate creation
Cryptographic signing	HMAC-SHA256 with tenant JWT secret; signature covers `mandate_id:customer_id:max_amount:expires_at`
Expiry enforcement	Mandates stored in KV with TTL matching `expires_in_hours` (default 24h); auto-deleted on expiry
Ownership verification	Customers can only view their own mandates; admin key required for cross-customer access
Amount validation	Checkout with mandate validates purchase total against `max_amount`
Scope restriction	Mandates can be scoped to `all`, a specific collection, or a specific product
Audit trail	All mandate events logged to `consent_records` and pushed to GA4
Tenant isolation	Mandates stored at `tenant:{shop}:mandate:{id}` — no cross-tenant access

---

Quick Reference

All credentials in one place. Use the Webflow App Config tab or `wrangler.toml` / `wrangler secret put`.

WHAT	WHERE TO SET	EXAMPLE
Worker URL	Webflow App: Config tab	`https://your-worker.workers.dev`
Xano API Base URL	Webflow App or `wrangler.toml`	`https://xxx.xano.io/api:XXX`
Xano API Key	Webflow App or `wrangler secret`	`eyJhbG...`
Google Client ID	Webflow App or `wrangler.toml`	`123456.apps.googleusercontent.com`
Google Client Secret	Webflow App or `wrangler secret`	`GOCSPX-xxx`
GA4 Measurement ID	Webflow App or `wrangler.toml`	`G-XXXXXXXXXX`
GA4 API Secret	Webflow App or `wrangler secret`	`xxxxxxxx`
Shopify Store Domain	Webflow App or `wrangler.toml`	`store.myshopify.com`
Shopify Shop ID	Webflow App or `wrangler.toml`	`64312475691`
Shopify Client ID	Webflow App or `wrangler.toml`	`890afa5e-...`
Shopify Admin Token	Webflow App or OAuth install flow	`shpua_xxx` (auto via install)
Shopify App Secret	Webflow App or `wrangler secret`	`shpss_xxx`
Webflow CMS Token	Webflow App or `wrangler secret`	`xxx...`
Webflow Collection ID	Webflow App	`xxx...`
Resend API Key	Webflow App or `wrangler secret`	`re_xxx`
Resend From Email	Webflow App or `wrangler.toml`	`Brand <noreply@domain.com>`
JWT Secret	`wrangler secret` only	`openssl rand -hex 32`

Default CRM Tags

These 17 tags are seeded when you initialize the tag system:

CATEGORY	TAGS
status	Active, Inactive
tier	VIP, Prospect
consent	Accepts Marketing, Rejects Marketing, Accepts TOU, Accepts Privacy, Accepts Cookie
marketing	Email Subscribed, Email Unsubscribed, SMS Subscribed
segment	High Value, Returning, New Customer, At Risk
campaign	Campaign

Tags added from the UCP dashboard that don't match a known category default to **campaign**.